МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ

по лабораторной работе №5

по дисциплине «Базы данных»

Тема: Тестирование БД на безопасность

Студент гр. 3341 _____ Мальцев К.Л.

Преподаватель _____ Заславский М.М.

Санкт-Петербург

2025

Цель работы.

Создать веб-сервер с SQL-инъекциями

Протестировать sqlmap

Найти уязвимости и исправить

Сделать скрытую уязвимость

Задание.

Вариант 9 Пусть требуется создать программную систему, предназначенную для диспетчера автобусного парка. Такая система должна обеспечивать хранение сведений о водителях, о маршрутах и характеристиках автобусов. Каждый водитель характеризуется паспортными данными, классом, стажем работы и окладом, причем оклад зависит от класса и стажа работы. Маршрут автобуса характеризуется номером маршрута, названием начального и конечного пункта движения, временем начала и конца движения, интервалом движения и протяженностью в минутах (время движения от кольца до кольца). Характеристиками автобуса являются: номер государственной регистрации автобуса, его тип и вместимость, причем вместимость автобуса зависит от его типа. Каждый водитель закреплен за отдельным автобусом и работает на определенном маршруте, но в случае поломки своего автобуса или болезни другого водителя может пересесть на другую машину. В базе должен храниться график работы водителей. Необходимо предусмотреть возможность корректировки БД в случаях поступления на работу нового водителя, списания старого автобуса, введения нового маршрута или изменения старого и т.п. Диспетчеру автопарка могут потребоваться следующие сведения:

Список водителей, работающих на определенном маршруте с указанием графика их работы?

Какие автобусы обслуживают данный маршрут?

Какие маршруты начинаются или заканчиваются в пункте с заданным названием?

Когда начинается и заканчивается движение автобусов на каждом маршруте?

Какова протяженность определенного маршрута?

Какова общая протяженность маршрутов, обслуживаемых автопарком?

Какие автобусы не вышли на линию, и по какой причине (неисправность, отсутствие водителя)?

Сделать простой web-сервер для выполнения запросов из ЛР3, например с express.js. Не обязательно делать авторизацию и т.п., хватит одного эдпоинта на каждый запрос, с параметрами запроса как query parameters.

Намеренно сделайте несколько (2-3) запроса, подверженных SQL-инъекциям

Проверьте Ваше API с помощью sqlmap (или чего-то аналогичного), передав эндпоинты в качестве целей атаки. Посмотрите, какие уязвимости он нашёл (и не нашёл), опишите пути к исправлению.

+2 балла, если напишете эндпоинт с уязвимостью, которая не находится sqlmap-ом.

Выполнение работы.

В 1-й лабораторной работе на основе текста задания была построена структура БД (рис.1).

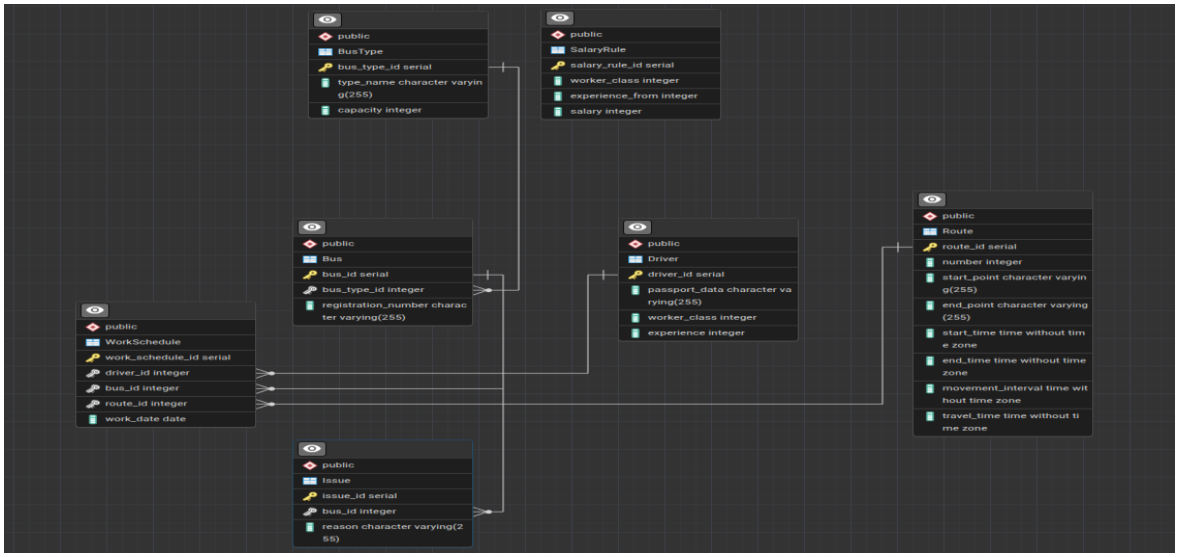

Рисунок 1 – Структура БД

Данная лабораторная работа была выполнена с использованием Sequelize.

Для удобства использется docker контейнер, где предустанавливаются нужные зависимости. Для работы можно подключиться к контейнеру по ssh.

Описание структуры:

- schema.js - описание моделей,
- seed.js - заполнение данными
- query.js — запросы
- server.js - сервер

Описание SQL запросов:

```
// SQL injection vulnerable
app.get('/unsafe/drivers', async (req, res) => {
   const result = await sequelize.query(`SELECT * FROM "Driver" WHERE experience = ${req.query.exp} LIMIT 10`);
   res.json(result[0]);
});

app.get('/unsafe/routes', async (req, res) => {
   const result = await sequelize.query(`SELECT * FROM "Route" WHERE start_point LIKE '%${req.query.search}%' LIMIT 10`);
   res.json(result[0]);
});

app.get('/unsafe/schedule', async (req, res) => {
   const result = await sequelize.query(`SELECT * FROM "WorkSchedule" ORDER BY ${req.query.order} LIMIT 10`);
   res.json(result[0]);
});
```

Разработанный программный код см. в **приложении А**.

Выводы.

Sqlmap успешно обнаружил базовые SQL-инъекции в GET-параметрах, но пропустил более сложные векторы атак через POST-запросы и JSON-данные, что демонстрирует необходимость комплексной защиты всех эндпоинтов через параметризованные запросы и валидацию входных данных независимо от типа HTTP-запроса.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

Ссылка на Pull-request с результатами выполненной лабораторной работы: https://github.com/moevm/sql-2025-3341/pull/39